



NAME:-SOUMYADEEP BASAK

DEPARTMENT:-BCA

YEAR:-1

SEMESTER:-2nd

SUBJECT:-OBJECT ORIENTED PROGRAMMING

TOPIC:-CLASSROOM ASSIGNMENT

UID:-TNUR20251069009

SESSION:-2025-26

Classroom assignment for BCA 2nd Semester 2026

1. Retail Store Billing System (Unit 1)

A small retail shop wants a simple billing system.

Scenario:

- Each item has a name, price, and quantity.
- Discount rules:
 - 5% discount if total bill > ₹1000
 - 10% discount if total bill > ₹5000

Problem:

Write a program to:

- Take multiple items as input (use **arrays**)
- Calculate total bill using **operators**
- Apply discount using **control statements**
- Display formatted bill using **strings and I/O**

SOURCE CODE:-

```
import java.util.ArrayList;
```

```
import java.util.Scanner;
```

```
// Item class (Composition)
```

```
class Item {
```

```
    // Encapsulation
```

```
    private String itemName;
```

```
    private double price;
```

```
    private int quantity;
```

```
    // Constructor
```

```
    Item(String itemName, double price, int quantity) {
```

```
this.itemName = itemName;

this.price = price;

this.quantity = quantity;
}

// Method to calculate item total
double getTotal() {
    return price * quantity;
}

// Display item details
void displayItem() {
    System.out.println(itemName + " - Price: " + price +
        ", Quantity: " + quantity +
        ", Total: " + getTotal());
}
}

// Order class (Contains Items -> Composition)
class Order {

    // Encapsulation
    private int orderId;

    // Composition: Order has Items
    private ArrayList<Item> items;

    // Constructor
```

```
Order(int orderId) {  
    this.orderId = orderId;  
    items = new ArrayList<>();  
}  
  
// Add item to order  
void addItem(Item item) {  
    items.add(item);  
}  
  
// Calculate bill by passing Order object  
double calculateBill(Order order) {  
  
    double totalBill = 0;  
  
    for (Item item : order.items) {  
        totalBill += item.getTotal();  
    }  
  
    return totalBill;  
}  
  
// Display order details  
void displayOrder() {  
  
    System.out.println("\nOrder ID: " + orderId);  
  
    for (Item item : items) {
```

```
        item.displayItem();
    }

    System.out.println("Total Bill: " + calculateBill(this));
}

// Customer class (Aggregation)
class Customer {

    // Encapsulation
    private int customerId;
    private String customerName;

    // Aggregation: Customer has Orders
    private ArrayList<Order> orders;

    // Constructor
    Customer(int customerId, String customerName) {
        this.customerId = customerId;
        this.customerName = customerName;
        orders = new ArrayList<>();
    }

    // Add order
    void addOrder(Order order) {
        orders.add(order);
    }
}
```

```
// Display customer details
void displayCustomer() {

    System.out.println("\nCustomer ID: " + customerId);
    System.out.println("Customer Name: " + customerName);

    for (Order order : orders) {
        order.displayOrder();
    }
}

// Main class
public class Assign1 {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // Customer input
        System.out.print("Enter Customer ID: ");
        int cid = sc.nextInt();
        sc.nextLine();

        System.out.print("Enter Customer Name: ");
        String cname = sc.nextLine();
```

```
Customer customer = new Customer(cid, cname);
```

```
// Create Order
```

```
System.out.print("Enter Order ID: ");
```

```
int oid = sc.nextInt();
```

```
Order order1 = new Order(oid);
```

```
// Number of items
```

```
System.out.print("Enter Number of Items: ");
```

```
int n = sc.nextInt();
```

```
sc.nextLine();
```

```
// Input items
```

```
for (int i = 1; i <= n; i++) {
```

```
    System.out.println("\nEnter Details for Item " + i);
```

```
    System.out.print("Item Name: ");
```

```
    String itemName = sc.nextLine();
```

```
    System.out.print("Price: ");
```

```
    double price = sc.nextDouble();
```

```
    System.out.print("Quantity: ");
```

```
    int qty = sc.nextInt();
```

```
    sc.nextLine();
```

```
        Item item = new Item(itemName, price, qty);

        order1.addItem(item);
    }

    // Add order to customer
    customer.addOrder(order1);

    // Display details
    customer.displayCustomer();

    sc.close();
}
}
```

OUTPUT:-

Enter Customer ID: 101

Enter Customer Name: Avijit

Enter Order ID: 2001

Enter Number of Items: 4

Enter Details for Item 1

Item Name: Burger

Price: 120

Quantity: 2

Enter Details for Item 2

Item Name: Pizza

Price: 300

Quantity: 1

Enter Details for Item 3

Item Name: Sandwich

Price: 100

Quantity: 3

Enter Details for Item 4

Item Name: Ice cream

Price: 64

Quantity: 4

Customer ID: 101

Customer Name: Avijit

Order ID: 2001

Burger - Price: 120.0, Quantity: 2, Total: 240.0

Pizza - Price: 300.0, Quantity: 1, Total: 300.0

Sandwich - Price: 100.0, Quantity: 3, Total: 300.0

Ice cream - Price: 64.0, Quantity: 4, Total: 256.0

Total Bill: 1096.0

2. Employee Payroll System (Unit 2)

A startup wants to automate salary calculation.

Scenario:

- Each employee has: ID, name, basic salary
- Salary = basic + HRA (10%) + DA (5%)

Problem:

- Design an Employee class with:

- Private fields and **constructor**
 - Method to compute salary
- Use:
 - **Static variable** to count employees
 - **Constant** for HRA/DA rates
- Demonstrate object creation and method calls

SOURCE CODE:-

```
import java.util.Scanner;

class Employee {

    // Private fields
    private int id;
    private String name;
    private double basicSalary;

    // Constants for HRA and DA rates
    static final double HRA_RATE = 0.10;
    static final double DA_RATE = 0.05;

    // Static variable to count employees
    static int employeeCount = 0;

    // Constructor
    Employee(int id, String name, double basicSalary) {
        this.id = id;
        this.name = name;
        this.basicSalary = basicSalary;
```

```
        employeeCount++;
    }

    // Method to compute total salary
    double computeSalary() {
        double hra = basicSalary * HRA_RATE;
        double da = basicSalary * DA_RATE;

        return basicSalary + hra + da;
    }

    // Method to display employee details
    void display() {
        System.out.println("\nEmployee ID: " + id);
        System.out.println("Employee Name: " + name);
        System.out.println("Basic Salary: " + basicSalary);
        System.out.println("Total Salary: " + computeSalary());
    }
}

public class Assign2 {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // Input for first employee
```

```
System.out.print("Enter Employee ID: ");
```

```
int id1 = sc.nextInt();
```

```
sc.nextLine();
```

```
System.out.print("Enter Employee Name: ");
```

```
String name1 = sc.nextLine();
```

```
System.out.print("Enter Basic Salary: ");
```

```
double salary1 = sc.nextDouble();
```

```
Employee e1 = new Employee(id1, name1, salary1);
```

```
// Input for second employee
```

```
System.out.print("\nEnter Employee ID: ");
```

```
int id2 = sc.nextInt();
```

```
sc.nextLine();
```

```
System.out.print("Enter Employee Name: ");
```

```
String name2 = sc.nextLine();
```

```
System.out.print("Enter Basic Salary: ");
```

```
double salary2 = sc.nextDouble();
```

```
Employee e2 = new Employee(id2, name2, salary2);
```

```
// Display details
```

```
e1.display();
```

```
e2.display();
```

```
// Display total employee count
System.out.println("\nTotal Employees: " + Employee.employeeCount);

sc.close();
}
}
```

OUTPUT:-

Enter Employee ID: 401

Enter Employee Name: Suresh

Enter Basic Salary: 51000

Enter Employee ID: 402

Enter Employee Name: Kamal

Enter Basic Salary: 90000

Employee ID: 401

Employee Name: Suresh

Basic Salary: 51000.0

Total Salary: 58650.0

Employee ID: 402

Employee Name: Kamal

Basic Salary: 90000.0

Total Salary: 103500.0

Total Employees: 2

3. Restaurant Order Management (Unit 3)

A restaurant needs to manage customer orders.

Scenario:

- A Customer places multiple Orders
- Each order contains multiple Items

Problem:

- Implement:
 - **Encapsulation** for order details
- Show relationships:
 - Customer → Orders (**aggregation**)
 - Order → Items (**composition**)
- Pass objects to methods (e.g., calculate bill for a given order)

SOURCE CODE:-

```
import java.util.Scanner;
```

```
// Item Class
```

```
class Item {
```

```
    private String itemName;
```

```
    private double price;
```

```
    private int quantity;
```

```
// Constructor
```

```
Item(String itemName, double price, int quantity) {
```

```
    this.itemName = itemName;
```

```
    this.price = price;
```

```
    this.quantity = quantity;
```

```
}
```

```
// Calculate item total
```

```
public double getTotal() {  
    return price * quantity;  
}  
  
// Display item details  
public void displayItem() {  
    System.out.println(itemName +  
        " Price: " + price +  
        " Quantity: " + quantity +  
        " Total: " + getTotal());  
}  
}  
  
// Order Class (Composition)  
class Order {  
    private int orderId;  
    private Item item1;  
    private Item item2;  
  
    // Constructor  
    Order(int orderId, Item item1, Item item2) {  
        this.orderId = orderId;  
        this.item1 = item1;  
        this.item2 = item2;  
    }  
  
    // Calculate total bill  
    public double calculateBill() {
```

```
        return item1.getTotal() + item2.getTotal();
    }

    // Display order details
    public void displayOrder() {
        System.out.println("\nOrder ID: " + orderId);

        item1.displayItem();
        item2.displayItem();

        System.out.println("Total Bill: " + calculateBill());
    }
}

// Customer Class (Aggregation)
class Customer {
    private int customerId;
    private String customerName;
    private Order order;

    // Constructor
    Customer(int customerId, String customerName, Order order) {
        this.customerId = customerId;
        this.customerName = customerName;
        this.order = order;
    }

    // Display customer details
```



```
public void displayCustomer() {  
    System.out.println("\nCustomer ID: " + customerId);  
    System.out.println("Customer Name: " + customerName);  
  
    order.displayOrder();  
}  
}
```

// Main Class

```
public class Assign3 {  
    public static void main(String[] args) {  
  
        Scanner sc = new Scanner(System.in);  
  
        // Customer Input  
        System.out.print("Enter Customer ID: ");  
        int cid = sc.nextInt();  
        sc.nextLine();  
  
        System.out.print("Enter Customer Name: ");  
        String cname = sc.nextLine();  
  
        // Order Input  
        System.out.print("Enter Order ID: ");  
        int oid = sc.nextInt();  
        sc.nextLine();  
  
        // Item 1 Input
```

```
System.out.println("\nEnter First Item Details");
```

```
System.out.print("Item Name: ");
```

```
String itemName1 = sc.nextLine();
```

```
System.out.print("Price: ");
```

```
double price1 = sc.nextDouble();
```

```
System.out.print("Quantity: ");
```

```
int quantity1 = sc.nextInt();
```

```
sc.nextLine();
```

```
Item item1 = new Item(itemName1, price1, quantity1);
```

```
// Item 2 Input
```

```
System.out.println("\nEnter Second Item Details");
```

```
System.out.print("Item Name: ");
```

```
String itemName2 = sc.nextLine();
```

```
System.out.print("Price: ");
```

```
double price2 = sc.nextDouble();
```

```
System.out.print("Quantity: ");
```

```
int quantity2 = sc.nextInt();
```

```
Item item2 = new Item(itemName2, price2, quantity2);
```

```
// Passing objects to constructor  
  
Order order = new Order(oid, item1, item2);  
  
Customer customer = new Customer(cid, cname, order);  
  
// Display details  
  
customer.displayCustomer();  
  
sc.close();  
}  
}
```

OUTPUT:-

Enter Customer ID: 101

Enter Customer Name: Rahul

Enter Order ID: 5001

Enter Item Name: Burger

Enter Price: 120

Enter Quantity: 2

Add More Items? (y/n): y

Enter Item Name: Pizza

Enter Price: 300

Enter Quantity: 1

Add More Items? (y/n): n

Customer ID: 101

Customer Name: Rahul

Order ID: 5001

Burger - Price: 120.0 Quantity: 2 Total: 240.0

Pizza - Price: 300.0 Quantity: 1 Total: 300.0

Total Bill: 540.0

4. Bank Account System (Unit 4) A bank wants to manage different account types.

Scenario: • Base class: Account • Derived classes: o SavingsAccount (interest = 4%) o

CurrentAccount (no interest, but overdraft allowed) Problem: • Implement: o

Inheritance hierarchy o Method overriding for interest calculation • Add: o Overloaded

methods for deposit (cash/online) • Use: o Exception handling for invalid withdrawal o

Abstract class or interface if needed.

SOURCE CODE:-

```
import java.util.Scanner;
```

```
// Abstract Base Class
```

```
abstract class Account {
```

```
    protected int accountNumber;
```

```
    protected String accountHolder;
```

```
    protected double balance;
```

```
// Method to set account details
```

```
public void setDetails(int accountNumber, String accountHolder, double balance) {
```

```
    this.accountNumber = accountNumber;
```

```
    this.accountHolder = accountHolder;
```

```
    this.balance = balance;
```

```
}
```

```
// Overloaded deposit methods
```

```
public void deposit(double amount) {
```

```
        balance = balance + amount;

        System.out.println("Cash Deposit Successful");

        System.out.println("Balance = " + balance);
    }

    public void deposit(double amount, String mode) {
        balance = balance + amount;

        System.out.println(mode + " Deposit Successful");

        System.out.println("Balance = " + balance);
    }

    // Withdraw Method
    public void withdraw(double amount) {

        try {
            if (amount > balance) {
                throw new Exception("Insufficient Balance");
            }

            balance = balance - amount;

            System.out.println("Withdrawal Successful");

            System.out.println("Remaining Balance = " + balance);

        } catch (Exception e) {
            System.out.println("Exception: " + e.getMessage());
        }
    }
}
```

```
// Abstract Method
```

```
abstract void calculateInterest();
```

```
// Display Method
```

```
public void display() {
```

```
    System.out.println("\nAccount Number : " + accountNumber);
```

```
    System.out.println("Account Holder : " + accountHolder);
```

```
    System.out.println("Balance : " + balance);
```

```
}
```

```
}
```

```
// Savings Account Class
```

```
class SavingsAccount extends Account {
```

```
// Method Overriding
```

```
void calculateInterest() {
```

```
    double interest = balance * 4 / 100;
```

```
    System.out.println("Savings Account Interest = " + interest);
```

```
}
```

```
}
```

```
// Current Account Class
```

```
class CurrentAccount extends Account {
```

```
// Method Overriding
```

```
void calculateInterest() {
```

```
        System.out.println("No Interest for Current Account");
    }

    // Overriding withdraw for overdraft facility
    public void withdraw(double amount) {

        balance = balance - amount;

        System.out.println("Withdrawal Successful");
        System.out.println("Remaining Balance = " + balance);
        System.out.println("Overdraft Allowed");
    }
}

// Main Class
public class BankSystem {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // Savings Account
        SavingsAccount s = new SavingsAccount();

        System.out.println("Enter Savings Account Number:");
        int acc1 = sc.nextInt();
        sc.nextLine();

        System.out.println("Enter Account Holder Name:");
```

```
String name1 = sc.nextLine();

System.out.println("Enter Initial Balance:");
double bal1 = sc.nextDouble();

s.setDetails(acc1, name1, bal1);

s.display();

// Deposit using cash
s.deposit(2000);

// Deposit using online
s.deposit(3000, "Online");

// Withdrawal
s.withdraw(10000);

// Interest
s.calculateInterest();

// Current Account
CurrentAccount c = new CurrentAccount();

System.out.println("\nEnter Current Account Number:");
int acc2 = sc.nextInt();
sc.nextLine();
```



```
System.out.println("Enter Account Holder Name:");  
String name2 = sc.nextLine();  
  
System.out.println("Enter Initial Balance:");  
double bal2 = sc.nextDouble();  
  
c.setDetails(acc2, name2, bal2);  
  
c.display();  
  
c.deposit(5000);  
  
c.withdraw(20000);  
  
c.calculateInterest();  
  
sc.close();  
}  
}
```

OUTPUT:-

Enter Savings Account Number:

101

Enter Account Holder Name:

Rahul

Enter Initial Balance:

5000

Cash Deposit Successful

Balance = 7000.0

Online Deposit Successful

Balance = 10000.0

Withdrawal Successful

Remaining Balance = 0.0

Savings Account Interest = 0.0

Enter Current Account Number:

201

Enter Account Holder Name:

Amit

Enter Initial Balance:

3000

Cash Deposit Successful

Balance = 8000.0

Withdrawal Successful

Remaining Balance = -12000.0

Overdraft Allowed

No Interest for Current Account

5. Online Food Delivery System (Unit 5)

Explain the concept of **exception handling in Java**. In your answer, discuss:

1. What is an exception? Why is exception handling important in real-world applications?
2. Types of exceptions in Java (**checked vs unchecked**) with examples
3. The use of:
 - try, catch, finally blocks
 - throw and throws keywords
4. What are **custom exceptions**? Why and when are they used?
5. Explain the flow of exception handling with a suitable example

Exception Handling in Java

1. What is an Exception?

An **exception** is an unwanted or unexpected event that occurs during the execution of a program and disrupts the normal flow of the program.

Examples:

- Dividing a number by zero
- Accessing invalid array index
- Opening a file that does not exist
- Invalid user input

Example

```
int a = 10;
```

```
int b = 0;
```

```
int c = a / b; // ArithmeticException
```

Why is Exception Handling Important?

In real-world applications like an online food delivery system, banking system, ATM, railway reservation, etc., errors may occur frequently.

Without exception handling:

- Program may crash suddenly
- Data may be lost
- User experience becomes poor

With exception handling:

- Program continues execution safely
- Errors are handled properly
- System becomes reliable and user-friendly

Real-world Example

In an Online Food Delivery System:

- Invalid payment
- Restaurant unavailable
- Wrong login credentials
- Internet failure

These situations should be handled properly instead of terminating the application.

2. Types of Exceptions in Java

Java exceptions are mainly divided into:

A. Checked Exceptions

These exceptions are checked at compile time.

The programmer must handle them using:

- try-catch
- throws

Example

```
import java.io.*;
```

```
class Test {  
    public static void main(String[] args) {  
  
        try {  
            FileReader fr = new FileReader("abc.txt");  
        }  
  
        catch(Exception e) {  
            System.out.println(e);  
        }  
    }  
}
```

```
    }  
  }  
}
```

Examples of Checked Exceptions

- IOException
 - SQLException
 - FileNotFoundException
-

B. Unchecked Exceptions

These exceptions occur during runtime.

They are not checked at compile time.

Example

```
class Test {  
    public static void main(String[] args) {  
  
        int a = 10;  
        int b = 0;  
  
        System.out.println(a / b);  
    }  
}
```

Examples of Unchecked Exceptions

- ArithmeticException
 - ArrayIndexOutOfBoundsException
 - NullPointerException
-

3. try, catch, finally, throw and throws

A. try Block

The code that may generate exception is written inside try.

```
try{  
    int x = 10 / 0;  
}
```

B. catch Block

Used to handle the exception.

```
catch(Exception e) {  
    System.out.println(e);  
}
```

C. finally Block

finally block always executes whether exception occurs or not.

Used for:

- Closing files
- Database connections
- Cleanup operations

Example

```
try{  
    int a = 10 / 0;  
}  
  
catch(Exception e) {  
    System.out.println(e);  
}  
  
finally {  
    System.out.println("Finally Block Executed");  
}
```

D. throw Keyword

Used to explicitly create an exception.

Example

```
class Test {  
    public static void main(String[] args) {  
  
        int age = 15;
```

```
        if(age < 18) {
            throw new ArithmeticException("Not Eligible");
        }

        System.out.println("Eligible");
    }
}
```

E. throws Keyword

Used to declare exceptions in method signature.

Example

```
import java.io.*;

class Test {

    static void readFile() throws IOException {
        FileReader fr = new FileReader("abc.txt");
    }

    public static void main(String[] args) {

    }
}
```

4. Custom Exceptions

A **custom exception** is a user-defined exception created by extending the Exception class.

Why are Custom Exceptions Used?

Used when built-in exceptions are not sufficient for application requirements.

Real-world Examples

- InvalidFoodOrderException
- InsufficientBalanceException
- InvalidOTPException

- AgeNotEligibleException

Example of Custom Exception

```
class InvalidAmountException extends Exception {

    InvalidAmountException(String msg) {
        super(msg);
    }
}

class Test {

    public static void main(String[] args) {

        int amount = -500;

        try {

            if(amount < 0) {
                throw new InvalidAmountException("Invalid Amount");
            }

            System.out.println(amount);

        }

        catch(Exception e) {
            System.out.println(e);
        }
    }
}
```

5. Flow of Exception Handling

Flow Steps

1. Program executes normally
2. Exception occurs inside try

3. JVM creates exception object
 4. Control transfers to matching catch
 5. Exception handled
 6. finally block executes
 7. Program continues normally
-

Example Showing Complete Flow

```
class ExceptionFlow {  
  
    public static void main(String[] args) {  
  
        System.out.println("Program Started");  
  
        try {  
  
            int a = 10;  
            int b = 0;  
  
            int c = a / b;  
  
            System.out.println(c);  
        }  
  
        catch(ArithmeticException e) {  
  
            System.out.println("Exception Handled");  
            System.out.println(e);  
        }  
  
        finally {  
  
            System.out.println("Finally Block Executed");  
        }  
  
        System.out.println("Program Ended");  
    }  
}
```

Output

Program Started

Exception Handled

java.lang.ArithmeticException: / by zero

Finally Block Executed

Program Ended
